1. **Comparatie intre modelul Gouraud si modelul Phong**

Modelul Gouraud

- Se integreaza foarte usor in algoritmii de rasterizare
- Calcule simple pentru culoarea unui fragment (calcul incremental, deci rapid)
- **Nu** permite calculul luminii reflectate specular pentru fragmentele interioare unei fatete (culorile fragmentelor interioare nu pot fi mai mari decat cele din varfuri)
- Nu elimina complet efectul de banda Mach:
- Asigura continuitatea numerica a valorilor intensitatilor la traversarea laturilor poligoanelor adiacente (culorile pe latura de adiacenta a 2 poligoane sunt aceleasi)
- Nu tine cont de tangentele la fetele adiacente pe o latura (culorile de pe laturi se calculeaza folosind numai culorile din varfuri)
- Efectul de banda Mach poate fi observat in vecinatatea siluetei suprafetei si a zonelor de curbura mare

Modelul Phong

La momentul rasterizarii, fiecarui varf al unui poligon îi este asociata normala în varf, care poate fi calculata la fel ca pentru modelul Gouraud. Normalele sunt calculate si asociate varfurilor în programul de aplicatie.

- Pentru fiecare fragment rezultat din rasterizarea unui poligon, GPU calculează o normala prin interpolare liniara intre normalele varfurilor.
- Culoarea pentru fiecare fragment interior poligonului se obtine pe baza normalei interpolate, folosind un model de iluminare locala

La rasterizarea unui poligon se calculeaza o normala pentru fiecare fragment astfel:

- prin interpolarea liniara a normalelor varfurilor, pentru fragmentele de pe laturi (la fel ca in modelul Gouraud, pentru culori);
- prin interpolare liniara intre normalele capetelor fiecarui segment interior, pentru fragmentele interioare poligonului (la fel ca in modelul Gouraud, pentru culori).

Dezavantaje fata de modelul Gouraud

- Componentele Nx, Ny, Nz ale normalei unui fragment se obţin printr-un calcul incremental (analog cu cel folosit pentru calculul culorilor in modelul Gouraud) dar, pentru folosirea în calculul culorii, normala trebuie să fie normalizată:
  - Nu = N/|Nu|, unde |N| = (Nx2 + Ny2 + Nz2)0.5
- De asemenea, vectorii L (catre sursa de lumina) si V (catre observator), folositi in modelul de iluminare local la nivel de fragment, sunt calculati si normalizati in programul fragment shader pentru a fi folositi in calculul culorii fragmentului.
- Calcule mai complexe la nivel de fragment decat in modelul Gouraud

Avantaje fata de modelul Gouraud

- Permite redarea reflexiei speculare in orice fragment al unui poligon
- Reduce mult efectul de banda Mach

2. Bressenham pentru rasterizarea vectorilor:

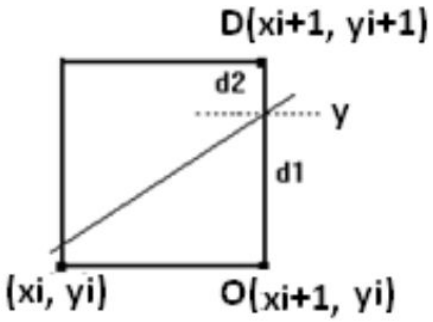
Alegerea punctelor spatiului discret.

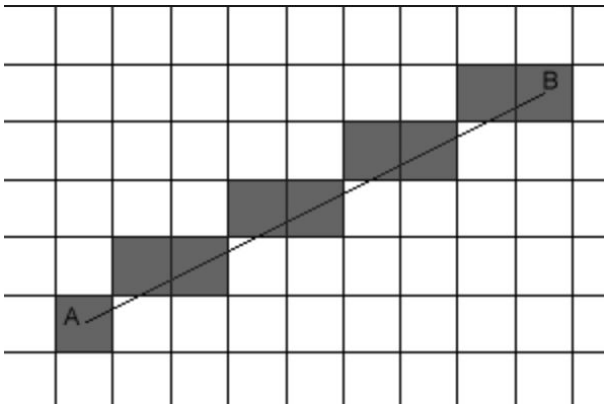
Contine numai operatii cu numere intregi. Este definit pentru vectori din primul octant

Un vector face parte din"primul octant" daca panta sa, m, satisface conditia 0<m<1.

Coordonatele pixelilor de pe traseul vectorului se obtin prin calcul incremental.

Pentru fiecare valoare a lui x, de la xmin_vector la xmax_vector, se alege acel punct al spaţiului discret care este mai apropiat de punctul de pe vectorul teoretic.





Fie

- m=(y2-y1)/(x2-x1) panta vectorului,
- (xi, yi) ultimul punct al spaţiului discret ales la executia algoritmului
- d1 distanţa de la punctul de pe vectorul teoretic, (xi+1, y), la punctul O(xi+1,yi)
- d2 distanţa de la punctul de pe vectorul teoretic la punctul D(xi+1,yi+1).

a.k.a. O = orizontala
D = diagonala

O si D sunt adrese de pixeli (puncte ale spatiului discret)

Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:

- O dacă d1 < d2, D în caz contrar.
- Dacă d1 = d2 se poate alege oricare dintre cele două puncte.

Deducerea expresiei de test.

Valoarea varibilei de test de obtine prin calcul incremental. Se adauga o constanta intreaga.

Eroarea de aproximare pentru primul pas:
xi = x1 şi yi = y1:
t1 = 2 * dy - dx

ti = (d1 - d2) * dx
= 2 * dy * xi - 2 * dx * yi + 2 * b * dx - dx + 2 * dy
(dx > 0)

(1) Dacă **ti <= 0**, se alege punctul O, deci
- xi + 1 = xi + 1
- yi + 1 = yi

Rezultă:
ti+1 = 2 * dy * (xi+1) - 2 * dx * yi + 2 * b * dx - dx + 2 * dy
sau
ti+1 = ti + 2 * dy

(2) Dacă **ti > 0**, se alege punctul D, deci
- xi+1 = xi + 1
- yi+1 = yi +1

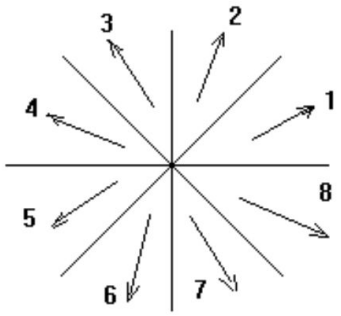
Rezultă:
ti+1 = 2 * dy * (xi+1) - 2 * dx * (yi + 1) + 2 * b * dx - dx +2 * dy
sau
ti+1 = ti + 2 * dy - 2 * dx

Implementare in C.

```
void Bres_vect(int x1, int y1, int x2, int y2) {
//pentru vectori cu panta cuprinsă între 0 şi 1
      int dx, c1, c2, x, y, t;
      dx = x2 - x1;
      c1 = (y2 - y1) << 1; // 2 * dy
      c2 = c1 - (dx << 1); // 2 * dy - 2 * dx
      t = c1 - dx; // 2 * dy - dx
      putpixel(x1, y1);
      for (x = x1 + 1, y = y1; x < x2; x ++) {
            if (t < 0)
                  t += c1; //deplasament O
            else {
                  t += c2;
                  y++;
            } // deplasament D
            putpixel(x,y);
      }
      putpixel(x2,y2);
}
```

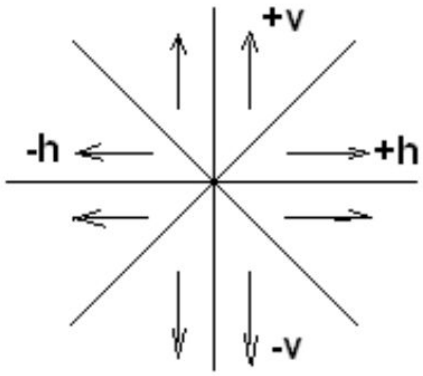
Generalizare pentru vectori de orice panta.

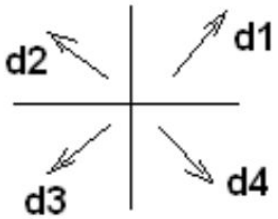
Generalizarea algoritmului Bresenham pentru vectori de orice panta.
Vectorii definiţi în planul XOY pot fi clasificaţi, pe baza pantei, în opt clase geometrice, numite "octanţi".
Un vector care apartine unui octant O are 7 vectori simetrici in ceilalti 7 octanti



dx=x2 – x1 şi dy=y2 – y1

- octantul 1: dx>0 şi dy>0 şi dx>=dy;
- octantul 2: dx>0 şi dy>0 şi dx<dy;
- octantul 3: dx<0 şi dy>0 şi abs(dx)<dy;
- octantul 4: dx<0 şi dy>0 şi abs(dx)>=dy;
- octantul 5: dx<0 şi dy<0 şi abs(dx)>=abs(dy);
- octantul 6: dx<0 şi dy<0 şi abs(dx)<abs(dy);
- octantul 7: dx>0 şi dy<0 şi dx<abs(dy);
- octantul 8: dx>0 şi dy<0 şi dx>=abs(dy);





Corespondenţa între alegerea curentă într-un pas al algoritmului Bresenham (punctul O sau punctul D) şi deplasamentul echivalent în fiecare octant:

Octant	1	2	3	4	5	6	7	8
Deplas O	+h	+v	+v	-h	-h	-v	-v	+h
Deplas D	d1	d1	d2	d2	d3	d3	d4	d4

3. Parametrii care definesc volumul vizual al proiectiei perspectiva in OpenGL. Planul in care se efectueaza proiectia.

Volumul vizual al proiectiei perspectiva (View frustum): trunchi de piramida delimitat de planul din faţă (near) si planul din spate (far) al vederii, cu varful in pozitia observatorului.
Se efectueaza proiectia perspectiva în planul din faţă (in fereastra vizuala).
Proiectia perspectiva este reprezentata de un trunchi de piramida (frustum) definit prin cele 2 planuri, **cel din apropiere** si **cel din departare**, cat si de deschiderea unghiurilor de vizualizare pe cele 2 axe, OX si OY.
Matricea de proiectie in acest caz poate fi calculata cu ajutorul functiei *glm::perspective* ce primeste ca si parametri

- deschiderea unghiului de vizualizare pe orizontala (**Field of View - FoV**)
- raportul dintre latimea si inaltimea ferestrei de vizualizare (**aspect ratio**)
- distanta pana la cele 2 planuri zFar si zNear.

4. Cum se construieste arborele BSP (scena 3D) si ce se memoreaza in noduri?

Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei). Fiecare plan de partitionare imparte spatiul in 2 semi-spatii:

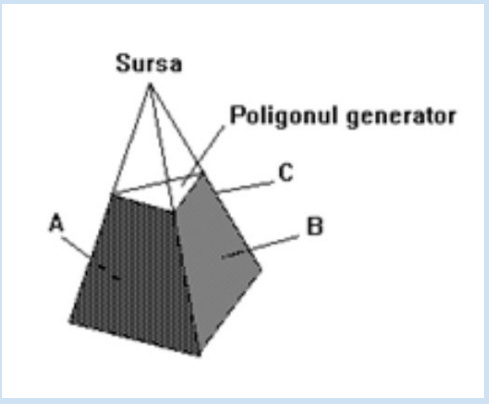
- cel din faţa planului (de aceeasi parte cu normala la plan)
- cel din spatele planului

Se incepe cu un poligon oarecare din lista (de regula primul), pentru care se creeaza nodul radacina al arborelui.

Primitivele din semi-spatiul "faţă" formeaza "lista-faţă", care va genera subarborele "faţă" al nodului.
Primitivele din semi-spatiul "spate" formeaza "lista-spate", care va genera subarborele "spate" al nodului.

Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui "faţă, etc.

5. Metoda volumelor de umbra (fara implementare)



- Cea mai populara metoda de introducere a umbrelor in imagini.
- Sursa de lumină este considerată punctiformă iar scena 3D alcatuita din poligoane.
- Un volum de umbră este definit de o sursă de lumină şi un poligon luminat (vizibil din poziţia sursei de lumină), pe care-l vom numi poligonul generator.
- Volumul infinit determinat de o sursă şi un poligon generator este delimitat de o faţă care reprezintă poligonul generator scalat. Această faţă este situată la o distanţă faţă de sursă dincolo de care intensitatea luminii sursei este neglijabilă, deci orice punct aflat dincolo de această limită este umbrit.
- Volumul de umbră poate fi decupat la marginile volumului vizual.
- Fiecare faţă laterală a volumului este numită poligon de umbră. Ea este determinată de o latură a poligonului generator şi de cele două drepte care pleacă din sursa de lumină, fiecare trecând printr-un vârf al laturii.
- Normalele la feţele laterale punctează înspre exteriorul volumului.

- Poligoanele de umbră se folosesc pentru determinarea umbririi produse de poligonul generator în scenă.

Notăm cu :

- PUV poligoanele de umbră care sunt vizibile din poziţia observatorului (A şi B în figura)
- PUN poligoanele de umbră care nu sunt vizibile din poziţia observatorului (de exemplu, poligonul C).

Clasificarea poligoanelor de umbra in PUV si PUN se poate face pe baza normalelor la poligoane (back face culling).

Fie

- un punct P al unei suprafeţe
- VP vectorul din poziţia observatorului (V) în punctul P

Punctul P este umbrit dacă numărul de poligoane de tip PUV intersectate de vectorul VP este mai mare decât numărul de poligoane de tip PUN intersectate de vector.
Acesta este singurul caz, atunci când punctul V nu este în umbră.

In general, pentru a determina dacă un punct este în umbră, se poate folosi un contor în care iniţial se memorează numărul de volume de umbră care conţin poziţia observatorului.

- Se asociază poligoanelor de tip PUV valoarea +1 iar celor de tip PUN valoarea -1.
- Atunci când vectorul VP traversează un poligon de umbră, se adună la contor valoarea asociată poligonului.
- Punctul P este umbrit dacă valoarea contorului este pozitivă în P.

Volumul de calcul presupus de acest algoritm poate fi redus dacă în loc să se calculeze volumul de umbră pentru fiecare poligon vizibil din poziţia sursei, se calculează un singur volum de umbră pentru o suprafaţă poliedrală. In acest scop, se determină poligoanele de umbră numai pentru laturile care fac parte din silueta suprafeţei, văzută din poziţia sursei de lumina.

Silueta unei suprafeţe, corespunzătoare unui punct de observare, este un set conectat de laturi care aparţin poligoanelor vizibile din punctul de observare.
O latură de siluetă este fie o latură de margine a unei suprafeţe deschise, fie o latură care separă un poligon vizibil de unul nevizibil.

Pentru determinarea laturilor de siluetă, este necesar să se folosească o structură de date care reflectă adiacenţa poligoanelor. Cunoscându-se poligonul adiacent pe fiecare latură a fiecărui poligon vizibil din poziţia sursei, se pot determina rapid laturile de siluetă.

Implementarea metodei folosind stencil-buffer

Buffer-ul de marcaje (stencil-buffer) este prezent pe placile grafice actuale (in memoria GPU), alaturi de buffer-ul imagine (frame-buffer) si buffer-ul de adancime (z-buffer).

- este o memorie de numere intregi asociata pixelilor imaginii, de regula un byte/pixel;
- poate fi utilizat pentru a implementa diferite operatii grafice, de regula in combinatie cu z-buffer, cum este si cazul implementarii volumelor de umbra:
- valorile din stencil-buffer pot fi incrementate/ decrementate automat pentru fiecare fragment care trece/ nu trece testul de adancime.

Exista mai multe metode de a utiliza stencil-buffer pentru implementarea metodei volumelor de umbre.

In metoda prezentata in continuare, se creaza in stencil-buffer o masca a umbrelor din imagine: orice pixel de umbra, (x,y), va fi reprezentat in stencil-buffer printr-o valoare ≠ 0.

Pixelii de umbra sunt afisati in culorile fragmentelor vizibile, calculate cu formula. Cfragment = Cemisiva + Cambianta = ke + ka*globalAmbient

ke, ka – proprietati de material

- Se presupune ca s-au calculat poligoanele de umbra.
- Conturul fiecarui poligon de umbra trebuie sa fie orientat in sens trigonometric atunci cand este privit din exteriorul volumului de umbra. In acest fel, poligoanele PUV sunt cele "din faţa" observatorului, iar poligoanele PUN sunt cele "din spate".

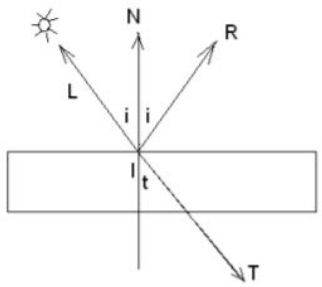
6. Ray Tracing

Care sunt razele folosite in calculul culorii unui pixel in algoritmul Ray Tracing?

L: raza din I catre sursa de lumina

R: raza reflectata specular, simetrica, fata de N, cu L

T: raza transmisa, calculata pe baza legii lui Snell:



Cum se calculeaza directia lor?

$R = 2(N \cdot L) \cdot N - L$

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refractie

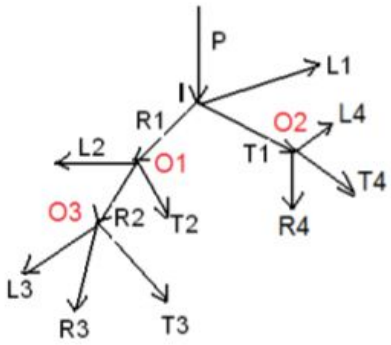
$T = L*(n1/n2) - (\cos(t) + (n1/n2)*(L \cdot N))*N$

Formula de calcul a culorii unui pixel?

Calculul culorii luminii în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks*R_\lambda(I) + Kt*T_\lambda(I)$$

Care este arborele de raze?



Arborele de raze

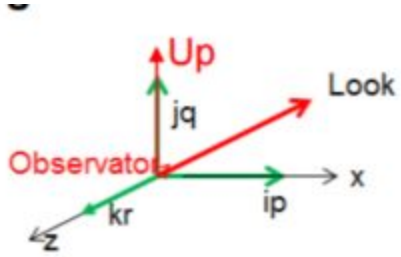
➢*Pentru obtinerea culorii pixelului, se evalueaza arborele de raze de la frunze catre radacina*

Cum se determina razele(fara exprimarea matematica)?

Se urmaresc cele trei tipuri L, R, T. Si daca o raza nu intersecteaza nimic se returneaza fondul altfel se aplica iar recursiv.

7. Parametrii care definesc sistemul de coordonate observator. Valori implicite in OpenGL. Functia care permite definirea acestora.

- poziţia observatorului (camerei): originea sistemului de coordonate globale
- direcţia în care priveşte observatorul: direcţia negativa a axei OZ
- direcţia sus a planului de vizualizare: direcţia pozitivã a axei OY

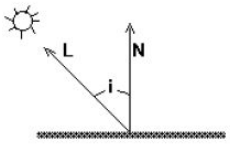


8. Reflexia difuza a luminii intr-un punct al unei suprafete 3D. Modelul empiric - sursa punctiforma.

Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D

- Lumina reflectată difuz de o suprafaţă este dispersată regulat în toate direcţiile.
- **Legea lui Lambert** defineşte reflexia luminii provenite de la o **sursă punctiformă**, de către un difuzor perfect:

$Id = Isursa * kd * \cos(i) \qquad 0 <= i <= \pi/2$



Reflexia difuza

Isursa – este intensitatea luminii incidente (provenita de la sursa de lumina)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafetei: suprafata reflecta/absoarbe/transmite lumina in anumite proportii: $0<=kd<=1$

Id este intensitatea luminii reflectate difuz de suprafata

- Dacă i este mai mare ca π/2, suprafaţa nu primeşte lumină de la sursă (sursa de lumină se află în spatele suprafeţei).

9. Cum se afiseaza o scena reprezentata printr-un arbore BSP?

- Tine cont de pozitia observatorului
- Afisare "back-to-front": se incepe cu primitiva cea mai indepartata de observator
- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in functie de pozitia observatorului fata de planul atasat fiecarui nod.
- In fiecare nod: se avanseaza in subarborele nodului care este opus observatorului fata de plan.

10. Functia de afisare a arborelui BSP.

```
void afisareBSP(arbore * A, Pozitie Observator) {
      if (!A) return;
      if (Observator este in semispatiul fata al planului radacinii){
            afisareBSP(A->spate);
            // radacina
            afisareBSP(A->fata);
      } else {
            afisareBSP(A->fata);
            // radacina
            afisareBSP(A->spate);
      }
}
```

11. Eliminarea poligoanelor nevizibile din banda grafica utilizand algoritmul BSP.

Daca planul de partitionare al unui nod nu intersecteaza volumul vizual, atunci numai subarborele aflat de aceeasi parte cu volumul vizual va fi afisat, celalalt subarbore fiind eliminat din banda grafica.

```
void afisareBSPcull (arbore * A, Pozitie Observator) {
	daca (! A) return;

	daca (Observator este in semispatiul fata al planului radacinii) atunci
		daca (planul nodului radacina nu intersecteaza volumul vizual) atunci
			afisareBSPcull(A->fata)
		altfel {
			afisareBSPcull(A->spate);
			*afisare primitive din nodul radacina;
			afisareBSPcull(A->fata);
		}
	altfel
		daca (planul nodului radacina nu intersecteaza volumul vizual) atunci
			afisareBSPcull(A->spate)
		altfel {
			afisareBSPcull(A->fata);
			*afisare primitive din nodul radacina;
			afisareBSPcull(A->spate);
		}
}
```

12. Cohen-Sutherland (decupare vectori 2D)

a) descriere

b) pasi pentru decuparea vectorilor din figura

13. Efectele proiectiilor perspectiva vs efectele proiectiilor axonometrice

Asemanari:

- Redau mai multe fete ale obiectului proiectat.
- Pot reprezenta pe un plan 2D un spatiu 3D.
- Unghiurile se conserva pentru fetele obiectului paralele cu planul de proiectie

Deosebiri:

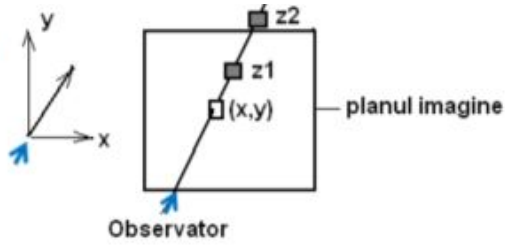
- La proiectiile perspectiva, proiectiile liniilor paralele care nu sunt paralele cu planul de proiectie converg catre un punct din plan, numit punct de convergenta; proiectiile axonometrice sunt afine - conserva paralelismul liniilor.
- Obiectele aflate la distanta mare se micsoreaza la perspectiva; in cazul proiectiilor axonometrice nu.

14. Z-buffer. Cand se executa. Depth-complexity.

- Algoritm de eliminare a partilor nevizibile ale primitivelor grafice.

- Executat de GPU, fiind **integrat in procesul de rasterizare** a primitivelor (dupa transformarea în poarta de afişare a vârfurilor)
- Observatorul este la infinit, pe axa z negativa
- Asupra primitivelor se efectueaza proiectie ortografica in planul XOY
- Doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

➢ Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Primitivele grafice (triunghiuri, linii) in care a fost descompusa scena 3D sunt rasterizate in ordinea in care au fost transmise din programul de aplicatie (nu sunt sortate). Buffer-ul imagine (frame buffer) este actualizat pe masura rasterizarii primitivelor.
Daca

- fragmentul curent se proiecteaza in pixelul (x,y)
- z-fragment < z-fragment afisat in pixelul (x,y)

=> actualizeaza culoarea pixelului (x,y) in bufer-ul imagine, la culoarea fragmentului curent

Observatie: este necesar sa se memoreze coordonatele z ale fragmentelor afisate in pixelii imaginii

Z-buffer:

- tablou bidimensional cu numar de elemente egal cu numarul de pixeli ai suprafetei de afisare
- în memoria placii grafice

Algoritm:

```
Initializeaza buffer-ul imagine la culoarea de fond
Initializeaza Z-buffer la coordonata z maxima (z = 1)
Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei
primitive
     calculeaza culoarea fragmentului
     //testul de vizibilitate (adancime)
     daca z < Z-buffer[y][x] atunci {
          Z-buffer[y][x] = z
     actualizeaza culoarea pixelului (x,y) in buffer-ul imagine
     folosind culoarea fragmentului f
```

}

Ultima transformare din lantul de transformari efctuate asupra varfurilor 3D conserva coordonata z a fiecarui varf.

Aprecieri asupra algoritmului z-Buffer

- Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel. Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.
- "Depth complexity": numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine.
- Calcul culoare fragment: model iluminare, utilizare texturi
- Pentru o precizie buna, este necesara o reprezentare pe 32 biţi a valorilor din z-buffer.

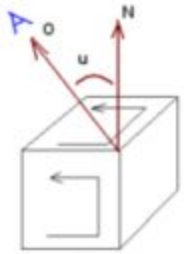
15. Determinarea fetelor autoobturate ale poliedrelor convexe. Conditia de vizibilitate atunci cand determinarea este efectuata: a) in coordonate globale; b) in coordonate de decupare.

Tip back face culling: Poate reduce substantial numarul de poligoane procesate in banda grafica.

Algoritm

- Observatorul si obiectul sunt raportati la acelasi sistem de coordonate.
- Conturul fiecarei feţe a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior.
- Observatorul este situat in afara obiectului. Faţa este vizibila daca : $0 <= u < 90^0$

N: normala la suprafaţa
O: vectorul orientat catre observator



Determinarea fetelor auto-obturate poate fi efectuata:

1. In sistemul **coordonatelor globale** (in care este definit si observatorul)

Produsul scalar:

$N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

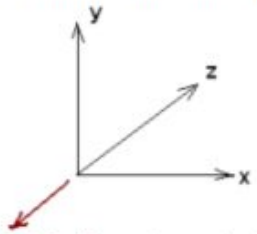
Conditia de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

2. In sistemul de coordonate observator

3. In sistemul **coordonatelor de decupare** (dupa aplicarea transformarii "model-view-projection")

3. In sistemul coordonatelor de decupare (dupa aplicarea transformarii "model-view-projection")

In cazul 3:



$Nu \bullet Ou = [nx, ny, nz] \bullet [0, 0, -1] = - nz$

Conditia de vizibilitate devine: nz <0

O[0, 0, -1] - este acelasi pentru orice punct al unui obiect

Observatorul este la infinit, pe axa z negativa.

16. Efectele proiectiillor oblice vs efectele proiectiilor axonometrice

La proiectiile oblice, proiectorii nu sunt perpendiculari pe planul de proiectie, pe cand la proiectiile axonometrice (care sunt proiectii ortogonale), proiectorii sunt perpendiculari pe planul de proiectie.
Astfel, in cazul unor proiectii oblice, in functie de unghiul din care este vizualizata raportul dintre lungimea laturilor difera.

17. Reflexia speculara a luminii intr-un punct al unei suprafete 3D. (modelul empiric - sursa punctiforma)

Modelul Phong pentru aproximarea reflexiei speculare intr-un punct al unei suprafete 3D:

$$Is\lambda = Isursa\lambda * w(i,\lambda) * \cos(\alpha)^n$$

w(i, λ) este funcţia de reflectanţă, i - unghiul de incidenţă iar λ - lungimea de undă a luminii incidente

In practica, w(i, λ) este înlocuită cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** $ks\lambda$.

- Modelul practic al reflexiei speculare:

$$Is\lambda = Isursa\lambda * ks\lambda * \cos(\alpha)^n$$

$\cos(\alpha)=R \cdot V / (|R| \cdot |V|) = Ru \cdot Vu$

Rezulta:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * (Ru \cdot Vu)^n$$

Pentru a include si cazul in care α = 90:

$$Is\lambda = Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$$

18. Ce transformare nu este afina?

Transformare perspectiva nu este o transformare afina deoarece ea nu conserva decat unghiurile din planul paralel cu planul de vizualizare. Este o transformare ce poate fi asemanata cu cea pe care o surprind aparatele de fotografiat.

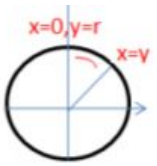
19. Cand trebuie sa fie reconstruit arborele BSP al unei scene?

Arborele nu trebuie modificat atunci cand se schimba pozitia observatorului, dar trebuie reconstruit atunci cand obiectele din scena isi schimba pozitia, pentru ca nu vor mai respecta relatiile pe care le au deja stabilite in cadrul BSP-ului.

20. Transformarea de vizualizare in OpenGL

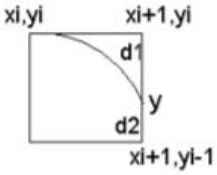
21. Bressenham pentru rasterizarea cercurilor. Puncte pe cerc, functia de afisare a cercurilor, implementare in C.

- Consideram cercul cu centrul în originea sistemului de coordonate si raza r.
- In cadrul algoritmului **se calculeaza numai punctele din octantul al 2-lea,** celelalte obtinandu-se prin simetrie.



- Se calculeaza punctele de pe cerc incepand cu (x=0,y=r) pana la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spatiului discret, ales pentru aproximarea cercului.

  Urmatorul punct va fi unul dintre (x_i+1,y_i) si (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 si d2 distantele de la cele 2 puncte ale spatiului discret la punctul de pe cercul teoretic.

Intereseaza semnul diferentei (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

Afisare

```
void Cerc_sim( int xc, int yc, int r, RGB culoare) {
      double pas, u, x, y, xx, s, c;
      // pas unghiular pentru obtinerea de puncte succesive de pe cerc
      pas = 1.0 / r;
      c = cos(pas);
      s = sin(pas);
      point (xc + r, yc, culoare);
      point (xc - r, yc, culoare);
      point (xc, yc + r, culoare);
      point (xc, yc - r, culoare);
      for (u = pas, x = r, y = 0; u < M_PI_4; u += pas) {
            xx = xx;
            x = xx * c - y * s;
            y = xx * s + y * c;
            punct_simetric ( xc, yc,
                             (int)(x+0.5),
                             (int)(y+0.5),
                             culoare);
      }
}
```

Implementare in C

```
void Bres_cerc(int xc, int yc, int r, RGB culoare) {
      int x, y, t;
      t = 3 - (r<<1);
      line (xc, yc + r, xc, yc - r, culoare);
      point (xc - r, yc, culoare);
      point (xc + r, yc, culoare);
      for (x = 1, y = r; x < y; x++) {
            if (t < 0)
                  t += 6 + (x << 2);
            else {
                  t += 10 + ((x - y) << 2);
                  y--;
            }
            // traseaza 4 linii simetrice
            line_simetric (xc, yc, x, y, culoare);
      }
      point (xc + x, yc + y, culoare);
      point (xc + x, yc - y, culoare);
      point (xc - x, yc + y, culoare);
      point (xc - x, yc - y, culoare);
}
```

22. Rasterizarea suprafetelor poligonale

- forma generala
- definitia punctului interior unui poligon
- cate puncte de intersectie se obtin cand o linie raster trece printr-un varf al poligonului?

Un punct este interior daca orice segment dus din el intersecteaza un numar impar de laturi. Pentru cele de maxim local nu se interseccteaza nicio latura, iar pentru cele de minim local se intersecteaza 2.

Se lua o linie raster. Se calculau toate intersectiile cu laturile, se determinau punctele de intersectie si se ordonau in functie de x. Si apoi se desenau segmentele.

23. Introducerea cetei in imagini :

a) *cum se modifica culoarea unui fragment tinand cont de factorul de ceata si culoarea de ceata?*

Fără ceaţă, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se aproprie de planul din spate şi deci este decupat atunci când planul din spate îl intersectează.

Cu ceaţă, dacă intensitatea ceţii creşte proporţional cu distanţa de la poziţia observatorului, se crează efectul de distanţă pentru obiectele îndepărtate.

Dacă densitatea ceţii creşte până la opacitate la o distanta mai mica decat aceea a planului din spate, decuparea faţă de planul din spate nu mai este necesară căci obiectele dispar datorită opacităţii.

Cfog–culoarea de ceaţă
Cfragment–culoarea calculată pentru un fragment
0<= Φ <= 1 – factorul de ceaţă, proporţional cu distanţa de la observator la punctul din spaţiul 3D vizibil in pixel (Φ=1: opacitate)
Atunci, Cpixel, culoarea de afişare a pixelului corespunzator fragmentului este:

Cpixel = (1 - Φ) * Cfragment + Φ * Cfog

b) *cum se calculeaza factorul de ceata?*

Factorul de ceaţă liniar (obtinut prin interpolare liniara) actioneaza intre 2 distante definite in sistemul coordonatelor observator, zo si z1, masurate pe directia de observare.
Factorul de ceata este calculat in timpul rasterizarii, pentru fiecare fragment, folosind coordonata z a fragmentului si distantele z0 si z1 transformate in coordonate ale spatiului de afisare:

- z: coordonata z a fragmentului
- Φ - Factorul de ceata
  - Φ = 0 pentru z < z0
  - Φ = (z - z0) / (z1 - z0) pentru z0 <= z <= z1
  - Φ = 1 pentru z > z1

Valorile implicite pentru z0 si z1 sunt 0 si 1, in spatiul de afisare: coordonatele z ale planului din fata si planului din spate al vederii, dupa transformarea in poarta de afisare.
Factorul de ceata moduleaza culoarea fragmentului dupa texturare.